

Coding & Solution

Date	27/06/2026
Team ID	Not Assigned
Project Name	Rising Waters: A Machine Learning Approach to Flood Prediction
Maximum Marks	5 Marks

Coding & Solution

This section evaluates the quality and completeness of the implemented code and the overall solution. The submission demonstrates working functionality, clean code practices, and alignment with the defined requirements for the Rising Waters flood prediction system.

Instructions:

1. Ensure your code is well-structured, commented, and follows best practices for your chosen technology stack.
2. The solution should address the core problem statement and implement the key functional requirements.
3. Include all source files, configuration files, and setup instructions needed to run the application.
4. Attach or link to your code repository (e.g., GitHub) in the table below.

Solution Summary

Field	Details
Repository Link / URL	https://github.com/Gowthamsai1234/rising-waters-flood-prediction
Live Application URL	https://rising-waters-flood-prediction.onrender.com
Programming Language(s)	Python 3.8+, HTML5, CSS3, JavaScript
Framework(s) Used	Flask 2.2+ (Web Framework), Gunicorn (WSGI Server), Jinja2 (Templating Engine)
Key Features Implemented	1. Exploratory Data Analysis (EDA) — distribution plots, box plots, correlation heatmap, violin plots 2. Data preprocessing — missing value handling, IQR outlier capping, label encoding, StandardScaler 3. Four ML classifiers trained and compared — Decision Tree, Random Forest, KNN, XGBoost 4. XGBoost model achieves 96.55% accuracy with class imbalance handling (scale_pos_weight) 5. Model saved as floods.save and scaler saved as transform.save using Joblib 6. Flask web app with 4 pages: home.html, index.html, chance.html, no_chance.html 7. Real-time flood prediction with probability score and advisory action recommendations 8. Dark-themed responsive UI with animated rain effect (main.css, main.js) 9. Application deployed on Render.com and accessible via public URL 10. Full codebase version-controlled on GitHub with automated deployment pipeline

Pending / Incomplete Features	1. Login / authentication system not implemented (not in project requirements) 2. Prediction history page not built (not required in functional specifications) 3. Real-time weather API integration not implemented (future enhancement) 4. SMS/email alert system not implemented (future enhancement)
Setup / Run Instructions	Step 1: Clone the repository git clone https://github.com/Gowthamsai1234/rising-waters-flood-prediction.git Step 2: Navigate to project folder cd rising-waters-flood-prediction Step 3: Install all dependencies pip install -r requirements.txt Step 4: Run the EDA notebook Open flood_eda.ipynb in Jupyter Notebook and run all cells Step 5: Train models and generate save files Open flood_model.ipynb and run all cells This generates: floods.save and transform.save Step 6: Start the Flask application python app.py Step 7: Open the application in browser http://127.0.0.1:5000 Step 8: Access live deployed application https://rising-waters-flood-prediction.onrender.com

Code Quality Checklist

S.No	Criteria	Status (Yes / No)
1	Code is modular and organized into functions / classes	Yes
2	Meaningful variable and function names are used	Yes
3	Code includes comments / documentation where necessary	Yes
4	Error handling is implemented for critical operations	Yes
5	The application runs without critical errors	Yes
6	Code is committed to a version control repository	Yes

Additional Notes / Comments:

The Rising Waters flood prediction system is fully implemented, tested and deployed. The XGBoost model achieves 96.55% accuracy on the test dataset, making it the best-performing model among the four classifiers evaluated. The Flask web application is live on Render.com and accessible globally. All source files, configuration files (Procfile, requirements.txt) and Jupyter notebooks are committed to the GitHub repository at https://github.com/Gowthamsai1234/rising-waters-flood-prediction. The application correctly handles all 10 meteorological input features, scales them using the saved StandardScaler, generates predictions using the saved XGBoost model, and displays the result with probability score and advisory recommendations on the appropriate result page.
